The Django ORM allows you to think of your data as objects, as instances of a Python class that you can access, modify, and delete rather than as rows in a database table. In the background, Django translates all this to the equivalent SQL code and runs it on your database server.

This also makes your interaction with the database agnostic to the actual database technology. Django supports MySQL / MariaDB, Postgres, SQLite or Oracle Database Server out of the box and can support others via third-party packages. It doesn't matter which one you use for the most part, the ORM will work in a similar manner.

Django's ORM has a powerful query syntax that allows you to search, sort and filter the contents of the database to get to exactly the entry or entries you want. It is also capable of efficiently combining multiple related tables, so it appears like you are using a single object while the data is coming from multiple tables. It also simplifies creating new database entries and updating exiting ones, whether one-at-a-time or in bulk.

Recent versions of Django can also track changes you make to your database structure during development, and will automatically generate code that will modify a database, taking it from the old structure to a new one without loss of data.

Django can also interface with multiple different databases at the same time in the same application.

Views

Django views are the bits of code that use the Django ORM among other things to create the response that the user will see when they browse a page.

Django supports two kinds of views. The old function-based views are simply a single function that can receives a request, crunches some data, and returns a response. The newer class-based views are more structured and offer common base-classes that encapsulate common functionality, such as displaying a single object, or listing objects of a particular type.

That isn't all when it comes to views, since they also need to handle data creation and editing. For instance you might have a registration form to let users sign up for a new account. Or you might have a contact page where users can leave you a message. These situations need you to access the data the user entered, verify and validate it and then add it to the database if needed.

Django provides tonnes of tools an utilities for all possible cases such as blocking users from accessing certain pages if they haven't logged in, or from posting data unless they have the correct permissions.